SAVEETHA SCHOOL OF ENGINEERING
C PROGRAMMING ASSIGNMENT – Grama Jyothi Billing Program

Field	Entry
Student Name	BAVADARANI S
Register Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Design, Implement and Validate a Reliable Billing Program
GitHub Repository URL	https://github.com/bavadarini28 /CSA0209-1912260562-BAVADARANI.S.git
Submission Date	12 NOVEMBER 2026

TASK 11 – Recommend and Defend the Final Design

Recommendation

Item	Recommendation (evidence)
Money	Integer paise; long long for totals (exact, Tasks 2, 7, 10)
Readings	unsigned short for 16-bit registers, differenced modulo 65536 (Task 2)
Units	unsigned int
Tariff	#define in one header (Task 3); one-line revision
Macros	Few, fully parenthesised; inline functions for anything with arguments
Debugging	#ifdef DEBUG, removed from release builds

Tariff revision: edit the macro (e.g. RATE2_PAISE 225 → 230) in tariff.h, recompile (gcc -Wall -Wextra) and re-run the test table; every bill changes consistently (Task 3).

Limitations: a revision needs recompiling and redistribution; only one tariff at a time; fixed three slabs; a 16-bit register assumption; at most one rollover per month; no categories beyond a code.

District scale: yes – with long long the monthly district total (~2.2×10^10 paise) uses about 0.24% of the 9.2×10^18 capacity, and the per-bill loop is O(1), so 800,000 bills take milliseconds.

Alternative Concept – tariff read from a configuration file

At start-up, fopen a text file such as "100 200 0 225 450 5000" and fscanf the t values into variables (see Task 12). Extra start-up time: O(t) to read t values (about 6–7 values – microseconds), once per run. Space: t variables, O(t) (≈ 56 bytes for seven 64-bit values). Benefit: no recompiling or redistribution on each revision; the board can change the file. Costs: the file must be validated (missing, non-numeric, negative values) and the constants lose compile-time type checking. For a tariff that changes occasionally and a program used by many offices, the file is better.

Result: The macro/paise design is correct and sufficient for the original problem; the file-based design is the natural next step.